MediChain: App Flow (v1.7, revised draft)

Oct 14, 2026 · @Abc

Purpose, scope, and conventions

This App Flow describes what each person sees and does in MediChain, screen by screen: how they arrive, where they can go, what each screen shows, and what happens when something goes wrong. The PRD says what the system does, the TRD says how it is built, and this document covers the experience layer between them.

> A patient should never have to read a hash to know who saw their records. (Design principle for this document)

Item	Value
Version	1.7, revised draft. Decisions D10 to D23 are accepted.
Date	14 October 2026 (v1.6 was dated 3 October 2026)
Companion documents	PRD v1.6, TRD v1.9, Backend Schema v1.2, Architecture v1.2
Team	Group 02, Department of CSE, BGS College of Engineering and Technology, Bengaluru
Course	Project Phase-I (BCS685). Confirm the label is still right for the current semester
Authors	Anush G, Punith A, Rohith Gowda R, Yagna Maurya M V

It is built from the PRD's UI requirements (UI-1 to UI-9) and the TRD's contract and service calls, so every screen names the real functions behind it. Doing that also exposed places where the UI needs something the PRD or TRD does not yet provide. Those are listed in Accepted resolutions rather than papered over.

Scope

- A responsive React web app on a local test network, per the PRD. No native mobile app.
- Guardian and custodian actions are scripted, not screens (decision D5, UI-7). They appear in the demo as scripts that drive the real recovery path.
- All data is synthetic (Synthea). The app says so on every screen.
- There is no issuer screen. Issuing a claim, the patient's Claim permission for the issuer, and withdrawing a claim are scripted in the demo.
- There is no push, email, or SMS notification. A patient sees a request only on the dashboard or the Activity screen after opening the app (PRD limitation 19).

Conventions

- SCR-nn is a screen. FLW-x is a flow, using the same letters as the PRD's key user flows (A routine sharing, B break-glass, C lost-key recovery, D cross-hospital) plus S for one-time setup. AF-n is a gap found while writing this document, now resolved.
- Rows and headings tagged *(v1.7)* are new or changed in this version. SCR-04, SCR-12, and SCR-13 were added after the PRD's first UI list, and the PRD now lists them as UI-8, UI-9, and INT-3.
- Plain language (UI-6): screens never show gas, raw keys, hashes, or share values. The glossary in Errors, states, and plain-language copy gives the words screens use instead.
- One demo configuration is used everywhere: four guardians plus the hospital backup, with three of the four guardians needed (n = 4, k = 3). Screen examples that show other numbers in v1.6 now match it.

Users and entry points

Six kinds of people reach the app, and each lands on one screen first.

Person	Arrives by	Lands on	Can reach
Patient	Connects their wallet. The first visit creates an account and a vault	SCR-03 Patient home	SCR-04 to SCR-10

Person	Arrives by	Lands on	Can reach
Clinician (doctor, lab, insurer, device)	Connects a wallet that holds one of those roles	SCR-12 Shared with me	Records shared with them
Emergency responder	Connects a wallet that holds the emergency-responder role	SCR-11 Emergency access	Only that screen
Guardian and institutional custodian	No screen. Scripted in the demo (D5, UI-7)	n/a	n/a
Second institution	Opens the Hospital B demo page (its own port, 5174)	SCR-13 Hospital B viewer	Only that page
Evaluators	Run the demo script and watch the screens above	See Demo walkthrough	Everything

Routing rules after connecting *(v1.7, D10)*

The v1.6 rule looked up the patient's identity under the connected wallet's address. A patient's identity is registered under their account, not their wallet, so that lookup came back empty for every patient. The rules now resolve the account first.

1. If the wallet is not on the local test network (chain ID 31337), show a single "Switch network" prompt and stop.
2. Resolve the wallet's patient account with `PatientAccountFactory.accountOfOwner(wallet)`. If an account exists and `PatientIdentity.getDilithiumKey(account)` returns a key, treat the wallet as a patient.
3. Check `RoleRegistry.hasRole` for the emergency-responder role, then for doctor, lab, insurer, and IoT device. Clinicians and responders register from their own address, so `getDilithiumKey(wallet)` identifies them.
4. If exactly one applies, go straight there. If more than one applies (a doctor who is also a patient), show "Continue as..." with the choices.
5. If none applies, show SCR-02. If no account exists yet, SCR-02 first offers to create one, and the relayer creates it and pays the gas. Then the person creates the vault.

Screen map and inventory

Thirteen screens and three scripts cover every journey. After SCR-01 routes a person, a patient reaches SCR-03 Patient home, and its cards open SCR-04 to SCR-10. A responder goes straight to SCR-11 and a clinician to SCR-12. SCR-02 is the detour for someone with no vault yet. The Hospital B viewer (SCR-13) is a separate demo page and is not part of this tree.

ID	Screen	Who	Main TRD calls	Source
SCR-01	Connect and route	Everyone	`accountOfOwner`, `getDilithiumKey`, `hasRole` *(v1.7)*	UI-6, PAT-7
SCR-02	Create or unlock your vault	Patient	`createAccount` through the relayer, `register`, key derivation from S *(v1.7)*	PAT-1, PAT-2, PAT-5 to PAT-8
SCR-03	Patient home	Patient	`capsuleStatus`, `getCommitments`, `getGuardianSet`, `getRequest`, `getRequestStatus`, AuditLogged events, `cancelRecovery` *(v1.7)*	UI-1, UI-5, BG-7
SCR-04	My records	Patient	`recordsOf`, IPFS fetch, signature check, decrypt	UI-8, DAT-2 to DAT-4
SCR-05	Share	Patient	`issueConsent`, key wrapping, `verifyEligibility`	UI-1, CON-4, DAT-5, ZKP-2
SCR-06	Who has access	Patient	`revokeConsent`	UI-1, CON-2, CON-3
SCR-07	Guardians	Patient	`registerShareCommitments`, `rotateShareCommitments`, `getGuardianSet` *(v1.7)*	UI-2, T1-2 to T1-6
SCR-08	Emergency info	Patient	`preAuthorizeCapsule`	UI-3, BG-4
SCR-09	Activity	Patient	AuditLogged events, signed audit records	UI-5, AUD-1, AUD-3

ID	Screen	Who	Main TRD calls	Source
SCR-10	Recover my access	Patient	`startRecovery`, coordinator (full mode), `completeRecovery`	T1-7 to T1-10
SCR-11	Emergency access	Responder	`triggerEmergencyAccess`, RecoveryCoordinator, `closeWindow`	UI-4, BG-1 to BG-6
SCR-12	Shared with me	Clinician	`canAccess`, `checkAccess`, `recordsOf`, retrieval path *(v1.7)*	UI-9, DAT-3 to DAT-5, DAT-9
SCR-13	Hospital B viewer	Second institution	The same calls as SCR-12, through a second endpoint	INT-1, INT-3, DAT-9
SCR-S1	Guardian responses (script)	Demo	Decrypt, re-wrap, submit a share	UI-7
SCR-S2	Custodian co-sign (script)	Demo	`custodianCoSign`	BG-6
SCR-S3	Demo runner (script)	Evaluators	Deploy, seed, trigger	BG-8, E2E-1

User journeys

Five journeys carry the product. FLW-S is one-time setup, FLW-A routine sharing, FLW-B break-glass access (the centerpiece), FLW-C lost-key recovery, and FLW-D cross-hospital access.

FLW-S One-time setup

Step	Screen	What the person does	What happens behind it
1	SCR-01	Connects a wallet	Network check and account lookup decide the route

Step	Screen	What the person does	What happens behind it
2	SCR-02	Creates a passphrase and presses Create	If no account exists yet, the relayer creates one and pays the gas *(v1.7, D10)*. The record secret S is created on this device and both key pairs are derived from it. The public keys are published to the identity contract from the patient's account (PAT-5 to PAT-8), and S goes into the passphrase-encrypted keystore
3	SCR-07	Chooses how many guardians and how many must agree, adds them, and adds the hospital backup. The demo uses four guardians, three of whom must agree	The app hands S to a one-shot dealer, wrapped to the dealer's one-time key. The dealer splits it into n+1 pieces and encrypts each for its holder, and the app stores only the checks on-chain (T1-2, T1-4). The screen warns when one kind of guardian would hold too many pieces *(v1.7, T1-5)*
4	SCR-08	Ticks which emergency details a responder may see and turns it on	The emergency info is stored and pre-authorized (BG-4)
5	SCR-03	Lands on a home screen showing all three set up	n/a

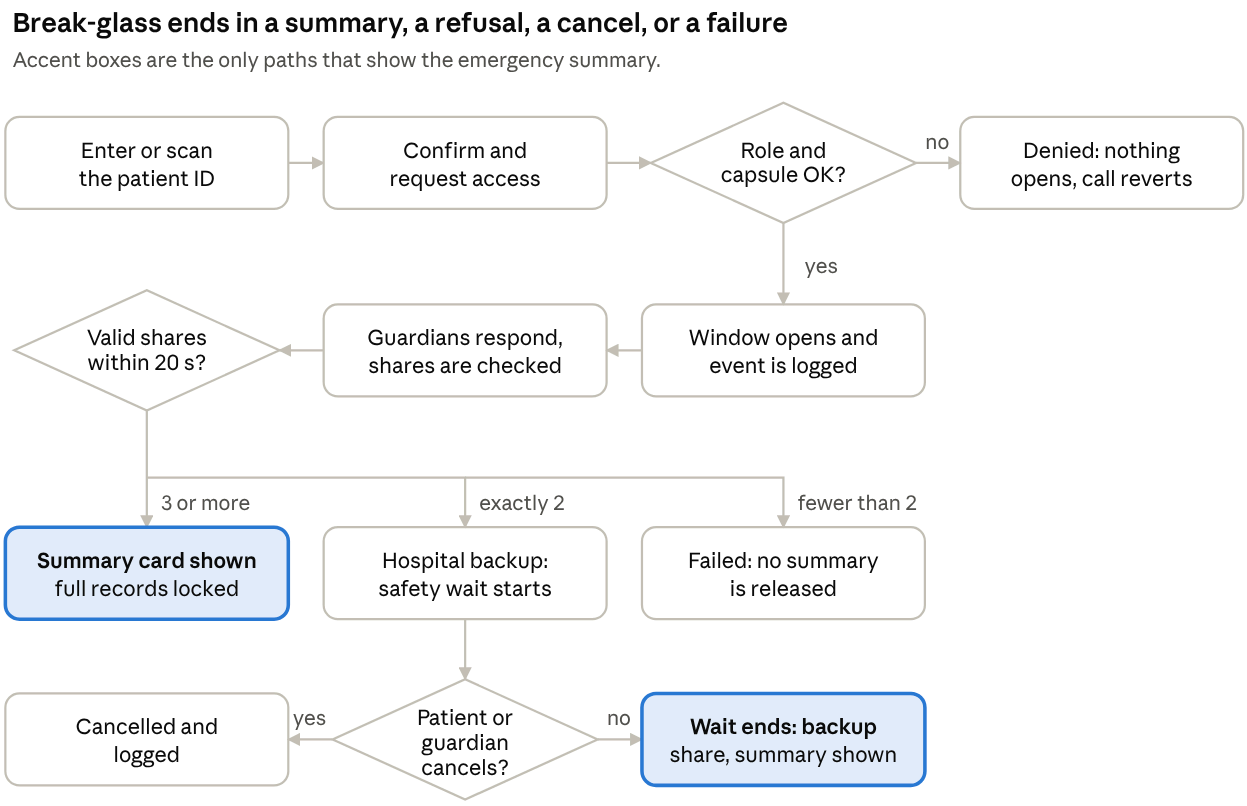
FLW-A Routine sharing

The patient shares, a clinician reads, the patient stops sharing, and the next attempt is denied. Every step is logged.

Step	Screen	Who	What happens
1	SCR-05	Patient	Picks the person, what to share, and for how long, then confirms. The permission is issued and a key for each chosen record type is wrapped for that person (CON-1, DAT-5)

Step	Screen	Who	What happens
2	SCR-12	Clinician	Sees the share under "Shared with me" and opens it. The app checks the permission with a free check and then a logged one, lists the records of that type from the record registry, fetches each one, verifies who created it, and decrypts it (CON-4, DAT-3, DAT-4) *(v1.7)*
3	SCR-05	Patient	For an insurer or pharmacy, optionally shares a proof instead of a record, such as "over 18". The proof is built in the browser from a claim a doctor or lab issued for this patient, and it is verified on-chain only against that issued commitment, once per request (ZKP-2, ZKP-3, ZKP-7). The verifier's request (claim type, threshold, request id) reaches the patient as a link, scripted in the demo, and the verified fact comes from the EligibilityVerified event, not from a consent token (D19)
4	SCR-06	Patient	Presses Stop sharing. The next attempt by the clinician is refused (CON-2). A key the clinician already holds cannot be taken back
5	SCR-09	Patient	Sees every open, share, and stop in Activity (AUD-1)

FLW-B Break-glass emergency access



break-glass flow · 3 decisions, 5 outcomes

The centerpiece flow, including the hospital-backup branch. The patient's cancel button lives on SCR-03. Every request that opens a window ends with the window closing and the expiry logged, whether it was released, cancelled, or failed.

Step	Screen	What the responder sees	What happens behind it
1	SCR-11	A role badge, then a field to enter or scan the patient's ID	The role was read at sign-in. If the role is missing, the screen shows Denied and stops
2	SCR-11	A confirmation: "I confirm this patient cannot give consent", then Request emergency access	`triggerEmergencyAccess(patientId)` checks the role and emergency info first, then opens the window and logs it in the same transaction (BG-2, BG-3, BG-5)
3	SCR-11	Stage 1 passes, and a countdown starts for the access window	The countdown is the window's expiry time minus the latest block time, not the browser clock alone *(v1.7, D11)*

Step	Screen	What the responder sees	What happens behind it
4	SCR-11	A list of guardians with Waiting, Responded, or Couldn't verify, and "3 of 4 needed" *(v1.7: v1.6 said "2 of 3")*	RecoveryCoordinator collects pieces, checks each against its on-chain check, and ignores any that fail (T1-7)
5a	SCR-11	The emergency summary card: blood type, allergies, medications, contacts. "Full records stay locked."	Only the pre-authorized emergency info is released (BG-4, T1-10)
5b	SCR-11	If exactly one guardian is short after 20 seconds: "Waiting for co-sign. Safety wait 00:42. This can be cancelled."	The hospital backup co-signs (SCR-S2) and the safety wait starts. The patient or any guardian can cancel it (BG-6, BG-7). After the wait the backup piece is released and the summary shows as in 5a
5c	SCR-11	If more than one guardian is short: "Not enough guardians responded in time."	No summary is released and the request ends
6	SCR-11	When the countdown ends the summary clears, and a line says emergency access has ended	`closeWindow` logs the expiry (AF-5)

FLW-C Lost-key recovery

The patient's own recovery path. It differs from FLW-B in who asks and what comes back: the secret, wrapped to this device only, not the emergency summary.

1. On a new device, the patient connects their account and chooses Lost your keys? Recover my access on SCR-02 (SCR-10). The app starts the request on-chain with `startRecovery`, signed through the patient's account, which stores a hash of a one-time key the new device made. It then sends that key to the coordinator, which only wraps the secret to a key matching the stored hash.
2. The app asks the guardians to respond. In the demo, scripted guardians reply.
3. Each genuine response ticks the progress bar. Invalid ones are shown as "couldn't verify" and ignored.

4. When enough arrive, the coordinator rebuilds the secret and hands it back wrapped to this device only, then wipes it. It never reads a record (T1-8, T1-10).
5. The app asks for a new passphrase and builds a new keystore. The signing and encryption keys are derived again, so the same public keys, every old signature, and every old wrapped key work, with no key change.
6. The app calls `completeRecovery`, which logs that recovery is complete, then prompts the patient to refresh their guardians' keys so any pieces exposed during recovery become useless (T1-6).
7. If exactly one guardian is short, the app offers the hospital backup with a visible safety wait. If more than one is short, it says recovery cannot finish yet.

FLW-D Cross-hospital access

1. The patient shares with a doctor at Hospital A (FLW-A).
2. On the Hospital B demo page (SCR-13), a user enters the same patient ID and permission.
3. Hospital B checks the permission without changing it, finds the records through the record registry, fetches them, verifies the creator, and decrypts them (INT-1).
4. The patient sees Hospital B's access in Activity.

Screen specifications

Each card lists what the screen is for, what it shows, what the person can do, the states it must handle, and the TRD calls behind it. Error wording is collected in a later section so it stays consistent. Cards SCR-01 to SCR-06 are below, and SCR-07 to SCR-13 and the scripts follow.

SCR-01 Connect and route *(v1.7)*

Item	Detail
Purpose	Connect the wallet, check the network, and send each person to the right place
Shows	A Connect button, a network status line, and the test-network notice

Item	Detail
Actions	Connect. Switch network. "Continue as…" when more than one role applies
States	No wallet found, wrong network, connecting, connected
Calls	`PatientAccountFactory.accountOfOwner(wallet)`, `getDilithiumKey`, `RoleRegistry.hasRole` (routing rules above)
Leads to	SCR-02, SCR-03, SCR-11, or SCR-12

SCR-02 Create or unlock your vault *(v1.7)*

Item	Detail
Purpose	First visit: create an account, an identity, and keys. Return visit: unlock the keys stored on this device
Shows	First visit: a two-line explanation, a passphrase field, and a progress list (setting up your account, creating your secret, deriving your keys, publishing public keys, done). Return visit: a passphrase field
Actions	Create. Unlock. "Lost your keys? Recover my access" goes to SCR-10. Optional: download an encrypted backup key file, or restore from one (AF-3)
States	Waiting for confirmation, done, wrong passphrase, already registered

Item	Detail
Calls	If no account exists, the relayer calls `createAccount(wallet)` and pays the gas (PAT-7, D10). The client creates S with `crypto.getRandomValues`, derives both key pairs from it, keeps only S in the passphrase-encrypted keystore, then calls `PatientIdentity.register(didHash, dilithiumPubKey, mlkemPubKey)` through the patient's account (AF-11). The DID hash is built from the account address (PAT-8). Unlocking re-derives the keys from S

SCR-03 Patient home *(v1.7)*

Item	Detail
Purpose	One-glance status
Shows	Four cards: **Who can see my records** (count and next expiry), **Emergency info** (On or Not set), **My guardians** (Not set up, "3 of 4 must agree", or Refresh due), and **Recent activity** (latest five)
Alert	While an emergency window or safety wait is live: "Emergency access was requested for you by Dr. Mehta. Cancel request". The alert appears only while the app is open. There is no push, email, or SMS (PRD limitation 19)
Actions	Each card opens its screen. Cancel request calls `RecoveryVault.cancelRecovery(requestId)`, which logs a cancellation (AF-6)

Item	Detail
Reads	`capsuleStatus` for the Emergency info card. `getCommitments` and `getGuardianSet` for the guardians card, because `updatedAt` drives Refresh due (errata 10). `getRequest` for a break-glass request and `getRequestStatus` for a lost-key safety wait. AuditLogged events for recent activity. The screen finds open request ids from the latest BreakGlassTriggered and RecoveryInitiated events *(v1.7)*
Empty state	A three-item checklist: add guardians, set emergency info, share your first record

SCR-04 My records

Item	Detail
Purpose	Let the patient see what is stored and who wrote each item
Shows	A list: plain-language title ("Blood glucose result"), date, "Added by you" or "Added by Dr. Rao", and a Verified badge once the signature checks out
Actions	Open a record (read-only). "Share this" jumps to SCR-05
Calls	`RecordRegistry.recordsOf(patientId, resourceType)` for each record type, then the IPFS fetch through the proxy, a signature check with the creator's published key, and decryption with keys derived from S (AF-13)
States	Empty, loading, and "We couldn't confirm who created this, so it isn't shown" when the signature fails. After a lost-key recovery the keys are re-derived, so earlier records still show Verified
Note	In the demo these come from the Synthea seed script (DAT-6)

SCR-05 Share *(v1.7)*

Item	Detail
Purpose	Give someone access that ends on the date you choose. Stopping later blocks future opens, but cannot take back a key they already hold
Step 1, Who	Enter their ID or pick a saved person. The app shows their role ("Dr. Rao, Doctor") and blocks anyone without one
Step 2, What	Choose record types in plain words, or choose **Prove a fact instead**, such as "I'm over 18". Only facts a doctor or lab has issued for you are offered. A proof is built in the browser from that issued value and works once per request, so the other side learns the fact and nothing else. If nothing has been issued yet: "No facts issued yet. A doctor or lab has to issue one first." Issuing is scripted in the demo: the scripted patient shares a Claim permission with the issuer first, and there is no issuer screen
Step 3, How long	1 day, 7 days, 30 days, or pick a date. It must be in the future
Step 4, Review	One sentence: "Dr. Rao can read your test results until 8 Oct. You can stop this at any time." Then Confirm
Calls	`ConsentRegistry.issueConsent`, sponsored through the relayer. The client wraps one type key per chosen record type to the grantee's ML-KEM key, and the keys go out in the `KeysWrapped` event (DAT-5, AF-15). For proofs, ProofService then `ProofGateway.verifyEligibility`, with the threshold the verifier requires and the request id
Leads to	SCR-06 with the new permission highlighted

SCR-06 Who has access

Item	Detail
Shows	Each permission: person and role, what, time left, and a status chip (Active, Ended, Stopped). Anything ending within 24 hours is highlighted
Actions	Stop sharing opens a confirmation: "They won't be able to open this again through MediChain. A copy or key they already have can't be taken back, and that includes records of this type you add later." That sentence is deliberate, since a delivered type key cannot be recalled
Calls	`ConsentRegistry.revokeConsent(tokenId)`. The list comes from `tokenIdsOf` and `getConsent` (AF-4)

SCR-07 Guardians *(v1.7)*

Item	Detail
Purpose	Choose who can help recover access, plus a hospital backup
Wizard	1. How many guardians, and how many must agree? The screen explains it as "3 of 4 guardians must agree. The hospital backup can stand in for one missing guardian." 2. Add each guardian with a group (Family, Hospital, or Guardian service). Each must already have a vault. The hospital backup row is required. 3. Review. 4. Create recovery pieces, with progress (making the pieces, locking one for each person, saving the checks) and a per-person Delivered or Waiting status

Item	Detail
Rules	At least 2 must agree, enforced on-chain. The screen recommends no more than the number of guardians (AF-12). If every guardian is in one group, warn: "All your guardians are family. Add someone from another group so one problem can't affect everyone." **New (D14):** also warn when any one group would hold as many pieces as must agree, for example "Three of your guardians are family, and three must agree. Spread them out so one group can't recover your records alone." The contract rejects both cases, so the warning comes first (T1-5)
After setup	A list with group badges and status, "3 of 4 must agree", the last refreshed date, and Refresh guardians, which re-deals the pieces so old ones stop working (T1-6). Refresh due shows once the last refresh is older than the policy interval. The patient starts every refresh. Nothing schedules one
Calls	The one-shot RecoveryDealer (the app hands it S wrapped to the dealer's one-time key through the local dealer launcher, and it overwrites what it can and exits, AF-16), `RecoveryVault.registerShareCommitments` or `rotateShareCommitments`, `getGuardianSet` for the refreshed date, and `PatientIdentity.getMLKEMKey` per person
States	A guardian has no vault yet: "They need to create a vault first." Dealer failure: "Nothing was saved. Try again." The contract refuses the set: "This mix of guardians isn't allowed. Spread them across groups."

SCR-08 Emergency info

Item	Detail
Purpose	Decide exactly what an emergency doctor may see
Shows	Four fields: blood type, allergies, current medications, emergency contacts, each with an include toggle. A live preview titled "What an emergency doctor would see", and a fixed line: "Everything else stays locked."
Storage	The emergency info is encrypted by the client library under its own key and registered with `RecordRegistry.addRecord` under the emergency-info type (AF-13). Save registers the capsule first and pre-authorizes it second, so a failure leaves a capsule nobody can release (D20)
Actions	Save and turn on calls `BreakGlassCapsule.preAuthorizeCapsule`. Turn off calls `revokeCapsule` and warns: "Emergency doctors won't be able to see anything." The On or Not set state comes from `capsuleStatus`
States	Not set, on (with last updated date), editing

SCR-09 Activity *(v1.7)*

Item	Detail
Shows	A reverse-chronological list in plain sentences: "Dr. Rao opened your test results", "Emergency access was requested by Dr. Mehta", "3 of 4 guardians responded", "Emergency access ended", "A record was added to your vault", "You started recovery", "Recovery completed". Filter chips: All, Sharing, Emergency, Recovery. Emergency events are coral

Item	Detail
Chips	**Sharing:** ConsentIssued, ConsentRevoked, AccessGranted, AccessDenied, ProofVerified, RecordAdded, ClaimIssued, ClaimRevoked. **Emergency:** BreakGlassTriggered, BreakGlassExpired. **Recovery:** RecoveryInitiated, RecoveryCompleted. TierEscalated and RecoveryCancelled go under Emergency for a break-glass request and under Recovery for a lost-key request. The mapping is the Backend Schema's, because v1.6 did not give one
Actions	Expand an event for who, what, when, and a "Signed record" check (AUD-3). The list updates live while the screen is open
Calls	AuditLogged events filtered by patient, with each ActionType mapped to a sentence. Sentences such as "3 of 4 guardians responded" come from the signed off-chain audit record, because no on-chain event carries them

SCR-10 Recover my access *(v1.7)*

Item	Detail
Entry	"Lost your keys?" on SCR-02
Shows	1. "Your guardians will help. At least 3 of 4 must agree." The app makes a one-time key on this device and opens the request with a hash of it. 2. A per-guardian list (Waiting, Responded, Couldn't verify) and "3 of 4 needed" *(v1.6 said "2 of 3")*. 3. When enough have responded: a prompt to choose a new passphrase for this device. 4. "Your records are unlocked and your identity is restored on this device", then "Refresh your guardians' keys now"

Item	Detail
If one short	"Your hospital backup can help. There is a safety wait first, shown as a countdown, and you or any guardian can stop it." It applies only when exactly one guardian is missing. The length comes from the safety-wait setting (60 seconds in the demo, per the TRD). If more than one is missing: "Recovery can't finish yet. Ask more guardians to respond."
Limit	This restores access to records, and your signing and encryption keys come back with it. It does not re-link a lost wallet (AF-10)
Calls	`RecoveryVault.startRecovery(patientId, clientKeyHash)`, the coordinator in full mode (the client sends the one-time key and receives S wrapped to it, and the coordinator only wraps to a key whose hash matches the stored one), `getCommitments`, `getRequestStatus` for the safety-wait countdown, `custodianCoSign`, `cancelRecovery`, and `completeRecovery` once the new keystore is unlocked (AF-17, AF-18)

SCR-11 Emergency access *(v1.7)*

Item	Detail
Purpose	Get critical information fast when a patient cannot consent, with every step visible and logged
Find the patient	Scan the code on their emergency card (it carries only the patient ID, `medichain://patient/...`) or enter the ID by hand (AF-8)

Item	Detail
Confirm	Checkbox: "I confirm this patient cannot give consent and this is an emergency." Then Request emergency access. A note says the request is recorded and the patient will see it the next time they open the app *(v1.6 said "the patient is told", but no notification exists)*. The tick is stored as an attestation in the signed audit record. The contract does not enforce it (AF-9)
Live timeline	1 Checking you're authorized. 2 Access window open, with a countdown. 3 Asking guardians, "3 of 4 needed", per-guardian status. 4 Hospital backup, only if exactly one is short: "Waiting for co-sign. Safety wait 00:42. This can be cancelled." 5 Emergency summary card, with "Full records stay locked." 6 Access ended
After expiry	The summary clears from the screen. A line says: "Emergency access has ended. This was logged." Information already seen cannot be taken back, so the screen says so plainly
Denied states	No responder role. No emergency info set. Cancelled. Not enough guardians in time. Window expired
Calls	`BreakGlassCapsule.triggerEmergencyAccess`, the coordinator's event stream on `127.0.0.1:4100` (AF-7), read with a fetch stream that sends the session token in a header (D17), `getRequest` for the countdowns, `getCommitments`, and `closeWindow` when the countdown ends (AF-5). Starting capsule mode also asks the responder's wallet to sign a challenge, so the coordinator can tie the request to the on-chain requester (D20)

SCR-12 Shared with me *(v1.7)*

Item	Detail
Why it exists	FLW-A needs a clinician to open what was shared, but the PRD's UI list had no clinician screen. Added (AF-1), reusing the responder screen's components
First step	Before anything else the screen shows "Unlock your keys" for the clinician's own keystore or, in the demo, "Demo identity loaded: Dr. Rao" (DAT-9, AF-19). In the demo the identity is one of Hardhat's unlocked accounts (D13)
Shows	Permissions where I am the grantee: patient, what, "ends in 6 days", Open. Proof-based shares show "Verified: patient is over 18" with no data. A record view renders the data readably, with a "Creator verified: Dr. Rao" badge and "Shared until 8 Oct, 10:00"
States	Ended or stopped shows "Access ended" and no data. A failed signature shows the SCR-04 message. No wrapped key shows "This wasn't shared with you."
Calls	`tokenIdsOfGrantee` and `getConsent` for the list. Then `canAccess` (a free check) and `ConsentRegistry.checkAccess` (a logged transaction from the funded demo account, with the outcome read from the receipt), `RecordRegistry.recordsOf`, the `KeysWrapped` event, and the rest of the TRD's authorized retrieval path
Stretch	Doctors and labs adding records (decision D9) can be seeded by script for the demo

SCR-13 Hospital B viewer *(v1.7)*

Item	Detail
Purpose	Show cross-system portability for INT-1 and INT-3
First step	Loads a demo identity for Hospital B from the demo fixture the seed script wrote, with no passphrase prompt in the demo (DAT-9, AF-19)
Shows	A "Hospital B (demo)" banner, a patient ID field, a permission field, and **Verify and open**. The result: "Permission is valid at Hospital B" and the same readable record. It runs as a second small app on port 5174 to make the separation visible (AF-2). It calls only the IPFS proxy's read-only fetch route and no other local service (D18)
States	Invalid, expired, or stopped permission
Calls	The same `canAccess`, `checkAccess`, and retrieval path as SCR-12, including `recordsOf` to find the CIDs, through a second endpoint

Scripts: SCR-S1 to SCR-S3

Script	Purpose and behavior
SCR-S1 Guardian responses	Drive the real recovery path without a guardian UI (decision D5, UI-7). For each scripted guardian: decrypt its own bundle, re-wrap the piece to the coordinator's one-time key, submit. Modes: `all-respond`, `one-tampered`, `one-short` (triggers the hospital-backup route), `timeout`. Each mode is a flag on the script, and it submits shares through `POST /requests/:id/shares` on the coordinator API with the request token (D22)
SCR-S2 Custodian co-sign	Calls `RecoveryVault.custodianCoSign` on cue, from the registered custodian's demo account, starting the safety wait (BG-6)

Script	Purpose and behavior
SCR-S3 Demo runner	`npm run demo:breakglass` starts the node with one-second block mining, deploys, seeds the Synthea data, and runs the scripted sequence (BG-8, E2E-1)

Errors, states, and plain-language copy

Twenty-four situations have fixed wording, so the same failure reads the same on every screen. Rows marked v1.7 are new.

Situation	Signal	What the screen says	Next step offered
Wrong network	Chain ID is not 31337	Switch to the MediChain test network.	Switch network button
No responder role	`triggerEmergencyAccess` reverts, "not authorized"	Your account isn't set up for emergency access.	None. Logged
No emergency info	Reverts, "no capsule"	This patient hasn't set up emergency info.	None
Permission ended	`isValid` is false	This access has ended.	Ask the patient to share again
Wrong record type	`checkAccess` is false	You don't have permission to see this type of record.	None
Proof failed	Verifier returns false	We couldn't confirm that. Nothing was shared.	Try again

Situation	Signal	What the screen says	Next step offered
No issued claim	`verifyEligibility` reverts, "no claim"	No fact like this has been issued for you, or it was withdrawn.	Ask a doctor or lab to issue it
Proof already used	Reverts, "nonce used"	This proof was already used. Make a new one.	Try again
Different requirement	Reverts, "wrong threshold" or "commitment mismatch"	This proof is for a different requirement, so it can't be used here.	Make a new proof
Not allowed to issue	`setCommitment` reverts, "not an issuer", "no claim token", "token names another issuer", or "token not from patient"	This person hasn't been given permission to issue facts for you.	Share a Claim permission with them first
Only the original issuer can replace	`setCommitment` reverts, "not the original issuer"	Another issuer already vouched for this fact, so only that issuer can change it.	Ask the original issuer
Bad guardian response	A piece fails its on-chain check	One response couldn't be verified and was ignored.	Wait for others
Not enough guardians	Timeout below the threshold	Not enough guardians responded in time.	Hospital backup if exactly one is short, otherwise ask more guardians
Safety wait running	Delay not elapsed	Safety wait in progress: 00:42.	Wait, or the patient cancels

Situation	Signal	What the screen says	Next step offered
Request cancelled	`cancelRecovery` called	This request was cancelled by the patient.	None
Window over	Past the expiry time	Emergency access has ended.	Request again if still needed
Unverifiable record	Signature check fails	We couldn't confirm who created this, so it isn't shown.	None
Storage unreachable	The IPFS fetch fails and the cache has no copy	We couldn't reach record storage. Try again in a moment.	Retry
Wallet declined	User rejects the prompt	You cancelled. Nothing changed.	Back
Invalid guardian numbers	Blocked in the UI and on-chain	Choose at least 2, and no more than your number of guardians.	Adjust
Guardians all one group	T1-5 check	All your guardians are family. Add someone from another group.	Add or continue anyway
Guardian mix rejected *(v1.7, D14)*	`registerShareCommitments` or `rotateShareCommitments` reverts for the category cap	This mix of guardians isn't allowed. Spread them across groups.	Adjust
Account setup failed *(v1.7, D10)*	`createAccount` through the relayer fails	We couldn't set up your account. Try again in a moment.	Retry

Situation	Signal	What the screen says	Next step offered
Local service unreachable *(v1.7, D17)*	The coordinator, dealer launcher, or IPFS proxy refuses the token or origin, or does not answer	We can't reach the recovery service on this computer.	Restart the demo, then retry

Words screens use instead of technical terms (UI-6)

Technical term	Screen wording
Consent token, issue consent	Permission, Share
Revoke	Stop sharing
Break-glass	Emergency access
Consent capsule	Emergency info
Guardian, trustee	Guardian: someone you trust to help you recover
k-of-(n+1) threshold	"3 of 4 guardians must agree", with the hospital backup standing in for one
Institutional custodian	Hospital backup
Delay window	Safety wait
Re-sharing, epoch	Refresh guardians
Zero-knowledge proof	Prove a fact without showing the record
Audit event	Activity
Patient account, factory	Your account. Set up for you, never shown as an address *(v1.7)*
Gas, hashes, share values, raw keys	Never shown
Wallet address	A name and role where known, otherwise a short ending such as "Af2a"

Cross-cutting UX rules

Eight rules apply to every screen, and two of them changed in v1.7.

1. **Test-network banner.** Every screen carries "Test network. Demo data only. No real patient information."
2. **One confirm step.** Every action that changes something shows one plain sentence and a Confirm button, then "Working…" and "Done". The patient signs a plain-language prompt and a relayer submits it, so no gas appears (AF-11). If the team drops that path, the wallet's own pop-up may show gas, and the app cannot hide it.
3. **Countdowns follow chain time.** *(v1.7, D11)* Show the time left as the stored expiry minus the latest block's timestamp, polled once a second, not from the browser clock. The demo node mines a block every second, so block time advances without transactions. The demo never advances the clock. Tests that do run on a separate network.
4. **Live updates.** Screens that show status subscribe to AuditLogged for the patient, with polling every few seconds as a fallback. The responder and recovery screens read the coordinator's stream with a fetch stream that sends the session token in a header.
5. **Honest limits in the copy.** Stopping a share cannot recall a saved copy or a key already given, and an emergency summary already on a screen cannot be unseen. Both say so. No screen promises a notification, because none exists.
6. **Clear on expiry.** Emergency information is removed from the screen the moment its window ends, and the coordinator drops it too.
7. **Speed on the responder screen.** Large buttons, high-contrast status chips, one primary action per step.
8. **Accessibility basics.** Status is never colour alone, since every chip also has an icon and text. Everything works by keyboard, and focus is visible. The layout works at laptop and phone widths.

Demo walkthrough

A suggested order for the viva, in twelve beats. It follows the PRD's advice to show failures as well as success, and it makes break-glass the centerpiece. Every beat uses the one demo configuration: four guardians plus the hospital backup, three of the four needed.

Beat	Screen or script	What the panel sees	What it shows
1. Starting point	SCR-03, SCR-07, SCR-08 (pre-staged by the seed script)	A patient with 4 guardians plus a hospital backup, 3 of 4 must agree, emergency info on, the demo identities loaded, the scripted patient having shared a Claim permission with Dr. Rao, and an age claim issued by Dr. Rao	Setup is done once and kept simple
2. Routine sharing	SCR-05, SCR-12, SCR-06	Patient shares with Dr. Rao. Dr. Rao opens it. Patient stops it. Dr. Rao is refused	Time-limited, revocable, role-scoped consent
3. Prove a fact	SCR-05, SCR-12	An insurer sees "over 18" and no birth date, and the same proof cannot be used a second time	Zero-knowledge proof of an issued claim
4. Emergency request	SCR-11	The responder requests access. Authorization passes, the window opens, a countdown starts	Role check, policy before cryptography, audit
5. Guardians respond	SCR-S1 (`all-respond`)	Responses tick up to "3 of 4 needed" *(v1.6 said "3 of 3")*	Threshold recovery
6. Emergency summary	SCR-11	Only blood type, allergies, medications, contacts. "Full records stay locked."	Capsule-only release
7. A bad response	SCR-S1 (`one-tampered`)	One response is refused before reconstruction	Commitment check

Beat	Screen or script	What the panel sees	What it shows
8. Hospital backup	SCR-S1 (`one-short`), SCR-S2, SCR-03	With two of four responding, the safety wait appears. The patient presses Cancel and the request stops. Run again and let it elapse	Tier 2 and the cancel right
9. Refusals	SCR-11	A user without the role is denied. A request after the window ends is denied	Access cannot be silent or open-ended
10. Patient's plain sentences	SCR-09	Every request, response, and closing in the Activity view	Auditability
11. Second hospital	SCR-13	The same permission opens the same record at Hospital B	Portability (INT-1)
12. Lost key (short version)	SCR-10	A new device recovers access with guardians, and the old records still show Verified	Recovery and a restored identity

Timing *(v1.7, D11)*

The TRD sets the demo values: a 180-second access window and a 60-second safety wait. v1.6 said beats 4 to 9 fit live and suggested advancing the network's clock to skip a wait. Both need correcting.

- Every trigger opens its own 180-second window, and beat 8 uses two requests. Beat 9 needs a window to end. Plan for roughly ten minutes for beats 4 to 9, and rehearse it to get a real figure.
- Start the beat 8 run that lets the wait elapse before you narrate beat 9, so a window ends while you talk.
- Do not advance the clock. Countdowns and the coordinator's deadlines read block time, and a jump would make them disagree. The demo node mines a block every second, so the clock moves on its own.

Accepted resolutions

Writing the screens against the PRD and TRD surfaced the first twelve gaps, and cross-checks added seven more. The team accepted all nineteen (AF-1 to AF-19), and the TRD specifies each one. The v1.7 review found nine more (AF-20 to AF-28), and the decisions they depend on are now accepted. A second review added five more (AF-29 to AF-33).

ID	Finding	Resolution	Where in the TRD	Owner	Status
AF-1	The PRD's UI list had no clinician screen, though FLW-A needs one	SCR-12 added, reusing the responder components	Frontend (ClinicianView)	Yagna	Accepted
AF-2	INT-1 had no UI, only a second endpoint	SCR-13 added: a minimal read-only page on port 5174	Frontend (HospitalBViewer)	Rohith	Accepted
AF-3	The patient's private keys and secret are held on the device, not on-chain, so a local store is needed	A passphrase-encrypted keystore in the browser, with an optional backup file and restore. The dealer hands S to it wrapped to a one-time key	Data schemas, Cryptographic specification, Frontend	Rohith, Yagna	Accepted
AF-4	No way to read a consent token, list tokens, or read request and capsule status	Added `getConsent`, `tokenIdsOf`, `tokenIdsOfGrantee`, `getRequest`, `getRequestStatus`, `capsuleStatus`, and `revokeCapsule`	Smart contract specifications	Anush, Yagna	Accepted
AF-5	A window expiring emitted no event	A permissionless `closeWindow(requestId)`, called by the responder screen or the coordinator when the countdown ends	BreakGlassCapsule, Recovery flow	Yagna	Accepted
AF-6	No audit type for a cancelled recovery	`RecoveryCancelled` added to `ActionType`, emitted by `cancelRecovery`	Data schemas, RecoveryVault	Punith, Yagna	Accepted
AF-7	No channel for per-guardian progress from the coordinator to the UI	A local HTTP and event-stream API on `127.0.0.1:4100`	Module interface contracts	Rohith, Yagna	Accepted
AF-8	No way for a responder to identify an unconscious patient	A scannable code on an emergency card or bracelet carrying the patient ID, with manual entry as a fallback	Data schemas (card payload), Frontend	Yagna	Accepted
AF-9	The confirmation checkbox on SCR-11 is UI-only	Recorded as an attestation in the signed off-chain audit record. The report does not claim the contract enforces it	Data schemas (signed audit record)	Yagna	Accepted

ID	Finding	Resolution	Where in the TRD	Owner	Status
AF-10	Lost-key recovery does not restore a lost wallet	Stated as a known limit in the report	Recovery flow, Known limits	Rohith, Anush	Accepted
AF-11	The wallet's own pop-up may show gas	Patient actions go through `PatientAccount.executeSigned` with a relayer paying gas. The fallback is direct wallet transactions	PatientAccount.sol, Frontend	Anush, Yagna	Accepted
AF-12	k = n+1 would always need the hospital backup, and guardian names are not on-chain	The UI recommends k no greater than n. Names live in a local address book, and after a device change the screen shows group and a short address	Frontend	Yagna, Rohith	Accepted
AF-13	No contract stored or listed records, so My records, Shared with me, and the Hospital B viewer could not find a patient's CIDs	A new `RecordRegistry` with `addRecord`, `recordsOf`, and `getRecord`. The CID is a string. Only the patient's account can add. The emergency info registers under its own type	RecordRegistry.sol	Anush	Accepted
AF-14	Lost-key recovery left the patient with new keys, so every old signature failed to verify	Both key pairs derive from the record secret S, which the client creates at vault creation. Rebuilding S restores the same keys	Cryptographic specification (Key hierarchy), T1-1 reworded	Rohith, Anush	Accepted
AF-15	Sharing is by record type, but keys were per record, so one key could not cover records added later	One type key per resource type, wrapped to the grantee and emitted in an event. A revoked grantee keeps a working key for later records of that type, and this is documented	Cryptographic specification, ConsentRegistry.sol, Known limits	Rohith, Anush	Accepted
AF-16	A browser cannot start the one-shot dealer	A local launcher on `127.0.0.1` with a per-session token. S reaches the dealer wrapped to a one-time key	Module interface contracts (dealer launcher)	Rohith, Yagna	Accepted
AF-17	The lost-key flow had no way to put S into the patient's new keystore	In full mode the client sends a one-time key and the coordinator returns S wrapped to it, then wipes S. The client builds a new keystore and re-derives its keys. The coordinator never decrypts a record, and only wraps to a key whose hash matches the stored one	Recovery flow (lost-key path), Module interface contracts	Rohith, Yagna	Accepted

ID	Finding	Resolution	Where in the TRD	Owner	Status
AF-18	No function let a patient open or complete a lost-key request on-chain	`startRecovery(patientId, clientKeyHash)` (the patient's account only) stores a hash of the new device's one-time key. `completeRecovery` closes it (the requester only, once). The coordinator checks the chain and the hash before wrapping S	RecoveryVault.sol	Rohith, Yagna	Accepted
AF-19	Clinicians, labs, and Hospital B also need a decryption key and a signing key, but the keystore held only the patient's	Each non-patient party holds its own keystore, with keys derived by the same HKDF scheme from its own secret, and registers its public keys like any party. The seed script creates the demo identities and writes their secrets to a git-ignored, demo-only fixture. SCR-12 and SCR-13 get an unlock or "demo identity loaded" step	Data schemas, Cryptographic specification, Frontend	Rohith, Yagna	Accepted
AF-20	No document said how a patient's account is created, and SCR-01 looked up the identity under the wallet, which is empty for every patient	A `PatientAccountFactory` creates one account per owner and resolves wallet to account. Routing resolves the account first	PatientAccountFactory.sol, Frontend (routing)	Anush	Accepted (D10)
AF-21	`checkAccess` both logs and returns a value, so a client cannot read it without sending a transaction	A free `canAccess` view for screens, and the logged `checkAccess` transaction for opens, read from the receipt. Demo identities are Hardhat's unlocked accounts	ConsentRegistry.sol, Frontend	Anush	Accepted (D13)
AF-22	Countdowns could freeze, and chain and coordinator clocks could disagree	One clock: one-second block mining, deadlines read block time, no clock jumps in the demo	Technology stack, Recovery flow (timing)	Yagna, Rohith	Accepted (D11)
AF-23	"Not all guardians in one group" lets three family members and one friend rebuild S at k = 3	A category cap in the contract and a matching warning on SCR-07	RecoveryVault.sol, Frontend	Rohith, Yagna	Accepted (D14)
AF-24	The browser's `EventSource` cannot send a bearer token, and cross-origin calls to the local services were not covered	A fetch-based reader sends the token in a header, and the services allow only the frontend origin	Module interface contracts (shared rules), Frontend	Rohith, Yagna	Accepted (D17)

ID	Finding	Resolution	Where in the TRD	Owner	Status
AF-25	Screens showed three different thresholds: "3 of 4", "3 of 3", and "2 of 3"	One demo configuration, n = 4 and k = 3, on every screen and beat	App Flow screens and demo walkthrough	Yagna	Accepted as wording
AF-26	SCR-11 said "the patient is told", but no notification exists	The copy says the patient sees it on opening the app. Limitation 19 added	PRD limitation 19, TRD Known limits	Yagna	Accepted as wording
AF-27	The documents did not say where record crypto runs, and the browser cannot hold the Pinata token	A client library does the crypto, and a loopback IPFS proxy holds the credentials and sees only ciphertext	Module interface contracts (IPFS proxy)	Rohith	Accepted (D12)
AF-28	Activity's filter chips had no mapping from each action	The Backend Schema's mapping is now in SCR-09	Data schemas (audit event)	Punith, Rohith	Accepted as wording
AF-29	Hospital B fetches ciphertext through the IPFS proxy, but only the frontend origin was allowed, and SCR-13 said it calls no local service	The proxy's read-only GET route also accepts the Hospital B origin. Pinning stays frontend-only. Hospital B calls no other service	IPFS proxy API, shared rules	Rohith	Accepted (D18)
AF-30	The proof-share flow had no request shape, and SCR-12 could not tell which fact it was showing, because a consent token has no claim type or threshold	A verifier's request names the claim type, threshold, and request id. ProofGateway emits EligibilityVerified, and the screen shows the fact from that event	ProofGateway.sol, proof request section	Punith, Anush	Accepted (D19)
AF-31	Capsule mode had no requester check, and saving emergency info did two transactions in no stated order	The responder signs a challenge, the coordinator checks the on-chain mode and window, and SCR-08 registers the capsule before pre-authorizing it	Capsule mode section	Rohith, Yagna	Accepted (D20)
AF-32	The hospital-backup co-sign had no guards, and the safety wait only holds because the coordinator honors it	The co-sign reverts for an unknown, cancelled, completed, or already co-signed request. Limitation 20 states that the wait is not cryptographic	RecoveryVault guards, Known limit 20	Yagna, Rohith	Accepted (D21)

ID	Finding	Resolution	Where in the TRD	Owner	Status
AF-33	SCR-S1 said guardians use the session token, while each coordinator request has its own token	The session token starts requests. A per-request token guards events, result, close, and shares, and a share must also be wrapped to the request's one-time key	Module interface contracts (tokens)	Rohith, Yagna	Accepted (D22)

Costs the screens and the threat model must be honest about

Stopping a share cannot take back a key a clinician already holds, and that covers records of the same type added later. Any k colluding guardians could forge the patient's signatures as well as read records. The coordinator process also briefly holds S while it works. There is no out-of-band notification, so a request is visible only after the patient opens the app. All four are in the TRD's Known limits, which match the PRD's Limitations to state in the report, and they belong in the threat model.

One correction touches the UI. `AuditLog.log` takes an `actor` argument, so Activity names the right person even when a relayer submits the transaction.

Traceability to the PRD

Ranges are inclusive. Together the rows cover all 67 requirement IDs in PRD v1.6. IDs with no screen say so and point to the TRD.

PRD requirements	Covered by
UI-1	SCR-03, SCR-05, SCR-06
UI-2	SCR-07
UI-3	SCR-08
UI-4	SCR-11
UI-5, AUD-1, AUD-3	SCR-03, SCR-09
UI-6	The plain-language table and the cross-cutting rules
UI-7	SCR-S1, with SCR-S2 and SCR-S3
UI-8	SCR-04

PRD requirements	Covered by
UI-9	SCR-12
PAT-1, PAT-2, PAT-5, PAT-6, PAT-7, PAT-8	SCR-01, SCR-02
PAT-3	SCR-02 and the one-confirm-step rule (sponsored actions)
PAT-4	SCR-01 routing, and the role checks on SCR-11 and SCR-12
CON-1 to CON-4	SCR-05, SCR-06, SCR-12
CON-5	No screen. The proof-verification hook is in the TRD
CON-6, INT-1, INT-3	SCR-13, FLW-D
ZKP-2, ZKP-3, ZKP-4, ZKP-7	SCR-05 (prove a fact), SCR-12 (verified facts), and the proof error rows
ZKP-1, ZKP-5, ZKP-6, INT-2	No screen. See the TRD
AUD-2, BG-3	No screen. Enforced in the contract, and visible in SCR-09
DAT-2 to DAT-4, DAT-8	SCR-04, SCR-08 (the emergency info registers as a record)
DAT-5	SCR-05, SCR-06 (the honest limit), SCR-12
DAT-1, DAT-6, DAT-7, DAT-10	No screen. DAT-6 shows as demo data on SCR-04 and in SCR-S3. See the TRD
DAT-9	SCR-12, SCR-13
T1-1 to T1-4	SCR-02, SCR-07
T1-5	SCR-07 (the group warnings)
T1-6	SCR-07 (Refresh guardians), SCR-10
T1-7, T1-8, T1-10	SCR-10, SCR-11, SCR-S1
T1-9, BG-5	SCR-11 (policy before any cryptography)
BG-1, BG-2	SCR-01 routing, SCR-11

PRD requirements	Covered by
BG-4	SCR-08, SCR-11
BG-6	SCR-10, SCR-11, SCR-S2
BG-7	SCR-03 (the alert and Cancel request), SCR-11
BG-8, E2E-1	SCR-S3 and the demo walkthrough
SEC-1, SEC-2	No screen. The costs note and the threat model
Non-functional: time handling	The countdown rule, SCR-11, SCR-10
Non-functional: local-service security	The event-stream rule, and the local-service error row